

# cmux-fleet

## Architecture, security and guardrails for an unattended local agent fleet

---

This document accompanies the briefing *Bounding the boundary*. It sets out how a fleet of containerised coding and operations agents is bounded in context, isolated from the host and from each other, and permitted to triage production without a human in the loop — together with the places where that design is currently incomplete.

### Scope and honesty note

Every quantitative claim here was read from the source tree at commit `1959b76` rather than recalled. Where a control is asserted by a test, that is stated. Where a control is *declared but not enforced*, that is stated too — there are four such cases and they are collected in §11 rather than scattered.

## Contents

---

- 1 The problem and the thesis
- 2 System shape: consoles, seats, providers
- 3 The bounded boundary
- 4 Token budget and admission control
- 5 Container isolation
- 6 Egress control
- 7 Credential handling
- 8 The verbgate
- 9 The five roles
- 10 Autonomous triage: the sweep
- 11 Known gaps and what they cost
- 12 What transfers to other platforms

Dan Elliott

## 1 · The problem and the thesis

---

A single coding agent consumes context at a rate set by the task. A *fleet* consumes it at a rate set by the fleet's size multiplied by whatever crosses the boundary between the host that orchestrates and the workers that execute. The interesting control point is therefore not the model, nor the prompt, but that boundary.

This matters for a reason that is easy to state and easy to miss in practice: an oversized brief does not run slowly, it *runs out*. Measured repeatedly across this fleet's own development, an agent handed too large a slice produces a complete, well-argued module, never wires it to a caller, stops mid-sentence, and reports success. The context ceiling lands on whatever is last in the queue — implement, then test, then integrate — so what it consumes is always the integration, which is the part with no unit test of its own to go red.

### The thesis

Bound the boundary in **bytes**, with an overflow policy chosen per direction; route the high-frequency traffic to **local** models; and confine paid inference to the two jobs that demonstrably benefit from frontier capability — code generation and review. Model selection then becomes a routing decision rather than a budget decision.

### Why not simply "measure the cost"

Because for this fleet the cost is not measurable, and pretending otherwise would be worse than not measuring. Ten of eighteen seats run on local weights, which have no price table. The system therefore reports `usd: 0` together with `priced: false` — an explicit anti-measurement whose entire purpose is to stop a downstream reader averaging `$0.00` into a spend report as though it were an observation.

A `usd_ceiling` concept existed and was *removed* rather than left in place unread, on the principle that a parsed-and-unread field advertises a control the product does not implement. Tokens are the only real ceiling, and they are mandatory.

The honest position to take to a finance partner is therefore not a saving but a **bound**: a hard cap on bytes per exchange, a token ceiling per run, and a majority of seats that cost electricity rather than API credit. Eight seats genuinely are billed, and the configuration says so in prose while attaching no number to it.

## 2 · System shape

The fleet is eighteen declared worker seats across four consoles. A console is a workspace of panes; the workers survive the window closing, so the console is a view rather than the system.

| CONSOLE     | SEATS                                    | CHARACTER                                                  |
|-------------|------------------------------------------|------------------------------------------------------------|
| operations  | obs-1, obs-2, sre-1, sre-2, tick-1       | read-only cluster work; ticketing holds a write credential |
| development | eng-1, eng-2, tst-1, tst-2               | own git worktrees; no egress for engineers                 |
| review      | col-1, rev-arch-1, rev-ctx-1, rev-lang-1 | read-only, shared mount, three vendors                     |
| triage      | tri-1, obs-t1, obs-t2, obs-t3            | <b>unattended</b> ; driven by a host-side actor            |

### Providers and the local/hosted split

| PROVIDER     | HOSTED     | TRANSPORT                  | SEATS |
|--------------|------------|----------------------------|-------|
| omlx         | no         | HTTPS tunnel to a LAN host | 4     |
| gabe         | no         | plain HTTP, LAN literal    | 6     |
| ollama-cloud | <b>yes</b> | HTTPS, billing credential  | 8     |

#### **hosted** is required and never inferred

The schema refuses to guess, and the reason is measured rather than stylistic: the local oMLX endpoint is HTTPS, publicly resolvable, on a public address. Every available heuristic would classify it as a hosted provider and therefore as a billing and data-egress concern. The declaration is the only reliable signal, so the schema makes it mandatory with an explicit error message.

The `gabe` provider is reached over plain HTTP rather than TLS, which is a deliberate and documented decision: corporate TLS interception breaks the tunnel route from inside the worker image. The traffic is confined to the LAN and the provider is declared `hosted: false`, so the trade is visible in configuration rather than hidden in a client.

### Attended versus unattended

Three consoles are driven by a person at a terminal. Triage is not: a host-side actor is both its clock and its fan-out performer, taking a per-console lock and running an unbounded loop at a configured cadence. Everything in §5 through §7 exists because that loop runs without anyone watching it.

### 3 · The bounded boundary

Four constants determine the maximum context any single host↔worker exchange can consume. They are enforced in the host, and *mirrored* inside the container image — the worker-side extension cannot import host source, so each constant is re-declared there and a test pins the two copies equal.

| CONSTANT                   | GOVERNS                     | VALUE   |
|----------------------------|-----------------------------|---------|
| MAX_DISPATCH_TEXT          | title and brief, per item   | 32 KiB  |
| MAX_DISPATCH_REQUEST_ITEMS | items in one request        | 8       |
| MAX_DISPATCH_REQUEST_BYTES | whole request file          | 4 MiB   |
| MAX_DISPATCH_POLICY_BYTES  | brief delivered to a worker | 256 KiB |
| MAX_REPLY_ARTIFACT_BYTES   | one returned file           | 64 KiB  |
| MAX_REPLY_INLINE_BYTES     | all artifacts in one reply  | 256 KiB |

The worst case is computed rather than assumed, and asserted by a test:  $8 \times 2 \times 32,768 \times 6 = 3.00 \text{ MiB}$  against the 4 MiB file cap — *"a document that validates is a document that reads"*.

#### Overflow policy differs by direction, deliberately

##### *Outbound: refuse*

An oversized request or brief is rejected whole, with a named refusal code, before anything is written to disk. A partially-delivered brief is a worse failure than a rejected one, because the agent cannot tell that it received half a task.

##### *Inbound: truncate, fairly, and say so*

Replies are allocated under **max–min fair allocation**. Each artifact receives an equal share of the 256 KiB envelope; any artifact under its share is paid in full and the surplus is redistributed among those over. The reason is recorded in capitals in the source:

ENUMERATION ORDER CANNOT DECIDE WHICH REVIEW SURVIVES

A naive "inline until the budget is gone" loop silently discards whichever reviewer was enumerated last. Under fair allocation a long review is trimmed, a short one untouched, and every trimmed artifact is *named in the collation brief* with both byte counts — so a truncated review cannot be read as a terse one.

##### **The escape valve is designed**

A long review is written to a file and declared as a **report** rather than pasted inline, and the host in-lines it under the two caps. The reviewer's own role prompt states the caps in bytes, so the constraint is visible to the agent making the decision rather than only to the host enforcing it.

The file cap is also checked twice — once from **fstat** before the read, again from the bytes read — because the worker owns the directory and can append between the two.

## 4 · Token budget and admission control

Each run carries a token ceiling — shipped as `6,000,000` with a per-task reserve of `400,000`. The ceiling is *required* with no default: its absence means unbounded, and inventing a default would refuse work no operator ever budgeted for.

### Admission is a reservation, not an estimate

Before a task is dispatched, the budget manager projects *spent + all outstanding reservations + this task's reserve* against the ceiling. Projecting against spend alone is the classic double-book: two tasks admitted back to back each individually fit and jointly overshoot. There are three named refusal reasons, checked in order:

| REFUSAL                     | CONDITION                                                        |
|-----------------------------|------------------------------------------------------------------|
| <code>budget_halted</code>  | the run is already halted; even a task that would fit is refused |
| <code>max_concurrent</code> | in-flight count at the concurrency limit                         |
| <code>would_exceed</code>   | the projection crosses the ceiling                               |

### Crossing the ceiling halts dispatch, not the world

When actuals are booked and the ceiling is crossed, the run is marked halted and exits with a dedicated budget exit code — but **artifacts are still harvested first**. The ceiling error is raised after harvest specifically so that a budget trip cannot destroy the evidence of what the tokens bought. Reservations are dropped on restart while the halt is carried forward, so a restarted actor cannot spend its way back over the line.

#### A consequence worth knowing

Because the last `per_task_reserve_tokens` of the ceiling can never be reserved, a run that uses its whole budget ends on an *admission refusal* rather than a crossed ceiling. That is the intended shape: the system declines to start work it cannot afford to finish, instead of discovering the problem afterwards.

### Where usage is actually recorded

Two sources were designed. One — a session-statistics RPC — is **not wired**, and cannot be for attended seats, which have no control plane to send it on. The live source is the transcript, folded per assistant message. For the ordinary dispatch path this works: the transcript is discovered by suffix-matching the worker id, and a failed lookup returns zero *with a degraded reason*, while the run's opening balance is floored at the last published snapshot so a failure cannot refund spend.

For the triage console specifically, this is broken. See §11.

## 5 · Container isolation

Every worker is a container running as uid/gid `10001`, with PID 1 supplied by `tini` and the agent binary pinned to an exact version in the image.

```
--user 10001:10001 --cap-drop ALL --security-opt no-new-privileges
--read-only (root fs) --pids-limit 512 --memory 4g --cpus 2
--tmpfs /tmp:rw,noexec,nosuid,size=256m --tmpfs /run:...,size=1m
--network <internal bridge> --env-file <mode 0600>
```

### The mount table is the capability model

| PATH                    | MODE | NOTES                                                               |
|-------------------------|------|---------------------------------------------------------------------|
| <code>/workspace</code> | rw   | a git <i>worktree</i> under the run directory                       |
| <code>/workspace</code> | ro   | when <code>isolation: shared-ro</code> — reviewers, collator        |
| (absent)                | —    | when <code>isolation: none</code> — triage has no code mount at all |
| <code>/skills</code>    | ro   | staged per role                                                     |
| <code>/policy/*</code>  | ro   | root-owned, 0555 — task provenance, allowlists, brief               |
| <code>/replies</code>   | ro   | the declared reply set                                              |
| <code>/secrets</code>   | ro   | one file per granted secret                                         |
| <code>/outbox</code>    | rw   | <b>the only writable route out</b>                                  |

### Three details that are easy to get wrong

#### *Guards run on the finished command line*

Mount assertions parse the actual argv, not source literals — three legitimate Docker mount spellings once walked past a literal-based check. A second variant resolves symlinks, since a link into the run directory is invisible in the unresolved path.

#### *No inline environment flags, asserted*

A test asserts the rendered argv contains zero `-e` flags and does contain `--env-file`. This keeps secret material out of the host process table, where it would be readable by any local user.

#### *Untrusted repository content cannot configure the agent*

Extensions, skills and context files are disabled *unconditionally* — a config key exists for compatibility but the flags are passed regardless, so the protection cannot be switched off. A scanner separately refuses repository state naming a program the version-control tooling would execute.

#### **Not present: a custom seccomp profile**

Containment rests on `--cap-drop ALL`, `no-new-privileges` and the platform default profile. A tailored syscall filter would be the next increment and is not currently implemented.

## 6 · Egress control

Network containment is three layers, and the second exists because the first has a hole that is not obvious from the documentation of the feature.

### Layer 1 — the internal bridge

Workers attach to a Docker network created `--internal`. Two properties are enforced rather than assumed: the host **refuses to adopt** a same-named network that is not internal, and re-inspects after creation rather than trusting the exit status.

### Layer 2 — the gateway block

`--internal` is a FORWARD-chain restriction, so the bridge *gateway address* stays reachable from inside the container. Not theoretical: a container on an internal network was observed pulling an SSH banner off the gateway. Closed with an explicit DROP rule.

#### It refuses the run rather than degrading

If the rule cannot be verified as installed, the run is refused. There is no override flag — deliberately, because an override on a containment control is the thing that gets used at three in the morning.

### Layer 3 — relay and CONNECT proxy

A dual-homed relay carries only literal `host:port` pairs for model traffic and runs an HTTP CONNECT proxy for everything else. The matcher is one implementation shared by both runtimes: wildcards match on **label boundaries** (so `evil-example.com` does not match `*.example.com`) and never match a bare IP literal. The default verdict is deny.

| RESPONSE | MEANING                                                 |
|----------|---------------------------------------------------------|
| 403      | policy denial, naming the rule                          |
| 405      | non-CONNECT request — cleartext proxying is not offered |
| 400      | no explicit port                                        |
| 431      | headers over 8 KiB                                      |
| 502      | allowed but unreachable                                 |

The proxy decides *before* it resolves: no DNS lookup and no socket is opened until the verdict is known, so a denied hostname is never contacted and never leaks into resolver logs. The relay is itself read-only, capability-dropped, IP-forwarding disabled, digest-pinned.

#### The grant is a route, not a destination set

A role's `egress_access` buys four proxy environment variables. The reachable host set is fleet-wide, so two workers with egress reach the same destinations. The source states the limitation directly: a fleet that needs one worker to reach a host another must not *has no way to express that*. See §11.

## 7 • Credential handling

Secrets are delivered as **files**, never as environment values. The environment carries only a pointer.

```
/secrets/TICKET_API_TOKEN          mode 0444, dir 0700
TICKET_API_TOKEN_FILE=/secrets/TICKET_API_TOKEN
```

Writes are verified by read-back — existence, type, exact length and mode — each failure raising a distinct error. A stricter 0400 was rejected because the container uid cannot read an operator-owned 0400 file; the tightening is bought one level up, on the directory.

### Scoping is an intersection with five refusals

A fleet-wide allowlist is the ceiling and a role's declaration is a draw against it. The refusals are explicit and throwing rather than filtering: not on the allowlist, missing from the host, a reserved name, a pointer collision, and a value containing a newline. Reserved namespaces cover the orchestrator's own prefixes and *every spelling* of the proxy variables, so a secret declaration cannot redirect a worker's egress.

### Cloud credentials are short-lived and stdin-delivered

Google credentials arrive as an access token written over `docker exec` with the value on **stdin**, **never argv**, into a tmpfs under a restrictive umask. Absence of a refresh token is *proved* by scanning all materials, and "no credential" is an explicit value with a reason rather than a missing field — so absence is testable.

### Redaction scrubs the serialised line

Not a list of known fields — the whole serialised event — precisely so that a new event shape cannot leak silently. Two details are worth transferring:

- **Leading-prefix matching.** A real incident ended with a model running `echo $TOKEN | head -c 20` — it truncated the secret *because it had been told not to echo it*. A whole-value matcher sees nothing; a prefix matcher catches it.
- **The marker names the variable**, so an operator reading `[redacted:TICKET_API_TOKEN]` knows what to rotate.

#### The limit, stated in the source rather than discovered

"A worker can still `cat` the file, and anything it cats lands in exactly the same transcript. That is a narrower blast radius, **not a seal**."

This is the right way to hold the control: file delivery converts an *accidental* disclosure into a *deliberate* one that must name the file. It does not prevent a determined or confused agent from reading its own credentials into a transcript.

## 8 · The verbgate

Credential scoping decides *which identity* a worker acts as; the verbgate decides *which verbs* it may issue, logging each to a ledger a host-side collector tails beyond the worker's reach. It is a set of shims on `PATH` ahead of the real binaries.

```
gcloud kubectl helm gsutil bq    -> the shim, first on PATH
/usr/local/libexec/<name>.real    -> the binary it fronts
```

| CHECK, IN ORDER    | OUTCOME                                                   |
|--------------------|-----------------------------------------------------------|
| policy writable    | exit 78 — refuses every verb, before classifying anything |
| read verb          | exec the real binary; ledger <code>allow_read</code>      |
| token-prefix match | exec; ledger <code>allow_listed</code>                    |
| anything else      | exit 77; ledger <code>refuse</code>                       |

### Two principles the shim states about itself

**A control the subject can reconfigure is not a control.** The paths are constants. They were once environment-overridable, which made the gate self-service: one variable supplied a policy file of the worker's own choosing, another pointed the ledger at `/dev/null` and discarded the evidence.

**A policy the subject can edit is not a policy.** Five surfaces are checked before classification. A file is tested directly *and through its parent*, because a writable directory lets a worker replace a read-only file wholesale; a directory is tested directly only. Matching is on verb tokens with flags excluded entirely — never a match against the whole command line, which reordering the flags defeats.

### Every way it failed open is recorded in the source

- A *positional* outvoted the verb — `gcloud compute instances delete real-vm list` classified as a read because an argument was named `list`.
- A glob minted credentials: `print-access-token` matched `print-*`, and `curl` is in the image — a bearer token buys any mutating REST call, ungated.
- The working directory decided the verdict — an unquoted argv of `*` re-expanded against the CWD, so a file named `describe` could reclassify a delete.
- The ledger dropped precisely the evasive rows: the argument terminator was missing, so any argv carrying a flag failed to serialise and the error suppression swallowed it.
- Provenance was forgeable — the task id came from environment, and a crafted value appended a duplicate `decision` key resolving in the worker's favour.

#### A capability that is refused is not the same as one silently unread

`cloud_allow[]` is designed but not built: the policy file is written empty and never rewritten, so every mutating cloud verb is refused today. The configuration layer *throws* on the key rather than accepting it — the deliberate opposite of §11's second gap, where a field is parsed and then consumed by nothing.

#### The limit, stated in the shim's own header

*"This stops the casual destructive action — the confused agent, the injected 'delete the failing deployment'. It does not stop a determined one: bash can reach the `.real` path directly. Credential scoping is the stronger control; this is defence-in-depth on top, **not a substitute**."*

## 9 • The five roles

Grants are declared per role and enforced at two points: the agent's tool registry is constructed with only the granted tools, and the configuration parser refuses combinations that contradict a declared `read_only` intent.

| ROLE      | BASH | WRITE | ISOLATION | DISTINCTIVE GRANT               |
|-----------|------|-------|-----------|---------------------------------|
| triage    | —    | —     | none      | <code>dispatch_request</code>   |
| observer  | yes  | yes   | none      | cloud + egress                  |
| ticketing | yes  | yes   | none      | egress + 4 secrets              |
| collator  | —    | —     | shared-ro | <code>dispatch_request</code>   |
| reviewer  | —    | —     | shared-ro | <code>submit_report</code> only |

Only the three bash-less roles genuinely forfeit write capability. A role holding `bash` can redirect to a file regardless, so removing `write` from a tester takes away a tool rather than a capability — the source says so explicitly, which is the kind of candour that makes the rest of the model trustworthy.

### Triage — and the lesson of the empty success

Triage holds no bash, no write, and has no code mount. Its product is a dispatch request file. When `write` was withdrawn without providing a verb to produce that file, the console broke *silently*: sweeps settled with `status: success` and the summary "Dispatched sweep to obs-t1" — having written nothing at all. The `dispatch_request` tool exists because removing a capability produced a confident, empty success rather than a failure.

### Observer — a turn budget, not a method

The observer is the only seat with live cluster access, restricted to read verbs by a wrapper that refuses any mutating verb not on an allowlist. Its dominant instruction is about *stopping*: "around twenty tool calls in, stop investigating and write what you have" — written after an observer made 150 correct calls against the right namespaces and ended its turn with an empty outbox.

### Ticketing — ask the server the question

It holds a live write credential and must verify every write by re-fetching the object. Its sharpest rule is about scale: never download a collection and search it locally. A run that enumerated 59,616 objects and grepped reported that a user with thirty open tickets had none.

### Collator and reviewer — manufactured independence

Three reviewer seats share an identical configuration and differ in exactly two fields: model and briefing file. Three *different vendors* is pinned by a test, because three seats on one model is one reviewer with three transcripts — a shared training blind spot becomes invisible by construction and independent agreement stops being independent. The collator may not form its own verdict first: "you cannot both pre-judge the change and be the one who says the agreement was independent."

## 10 · Autonomous triage: the sweep

The triage console runs unattended on a cadence. One pass is called a sweep, and it is the clearest worked example of the whole design.

1. **Lock.** The actor takes a per-console lock and refuses if another holds it — two actors sweeping one control plane is precisely the overload the console exists to detect.
2. **Mint.** A sweep id is `max(record, run tree)`, run tree authoritative. No id is reused; a resumed actor re-deriving a dispatched id is refused.
3. **Partition.** The collator receives the *whole* declared service list and divides it; the host validates the union covers each service **exactly once**.
4. **Fan out.** Observers dispatch *concurrently* against one deadline, so a sweep costs the largest share rather than the sum.
5. **Join, collate, assess, announce.**

### Concurrency is an anti-criterion, not a preference

Serial dispatch once meant one observer received its slice and the other two received nothing before the pass ended. The sweep then reported two services it "could not see", pointing an operator at a cluster that was healthy and reachable. *The cost of getting this wrong is not throughput, it is coverage.*

An incomplete or duplicated partition refuses the **whole** sweep — nothing is dispatched. A partial sweep that presents as complete is worse than no sweep, because it produces a clean report for services nobody looked at.

### Three gates that make an unattended verdict trustworthy

- **One row per declared service**, not per row the worker wrote. A service the agent omitted still appears, marked indeterminate; a service it invented cannot appear at all.
- **An unevidenced "healthy" is downgraded** to indeterminate. There is no exported code path that hands a caller the worker's raw health claim.
- **Coverage is counted host-side** from the run tree, never from the worker's account of what it did.

The single worker-authored string reaching a notification is fenced between banners and line-prefixed, so no agent output can be mistaken for a host statement. The agent is an **untrusted reporter** throughout: its prose is accepted, its counts never are.

### A worked failure, and its fix

The in-flight predicate asked whether a sweep had *settled* and never how long it had *owed*. A collator whose turn died mid-work left a sweep permanently "in flight", and the actor skipped eleven consecutive passes over nearly three hours until the console was rebuilt by hand. The console *did* report itself — a skip alarm fired at the third skip and then deduplicated into silence, which is right for a service that is down and wrong for the monitor being dead.

The fix separates fact from policy: the predicate reports *when* the sweep was dispatched; the pass, which already holds the clock and the configuration, decides what is too old. The bound is `max(max_consecutive_skips × cadence, 2 × sweep_deadline)`, keeping the existing alarm as the early warning rather than silently retiring it. A sweep whose envelope cannot be dated is never expired.

## 11 · Known gaps and what they cost

These are collected rather than scattered, and each is recorded in the source at the point where it would otherwise mislead a reader.

### 1 · The triage console's token ceiling cannot trip

Its per-seat accounting reads a worker-state usage field, to avoid pulling the harvest module into a deliberately narrow import closure. Nothing ever writes that field, so the accounting sums zeros across four seats and the ceiling is structurally unreachable here. Worse, a readable-but-default state file returns zero rather than "unknown", so the seat is not flagged degraded and the floor-at-last-snapshot protection never engages.

**Cost:** the one console that runs unattended and indefinitely is the one whose budget ceiling does not apply. **Fix:** read the transcript here as the dispatch path already does.

### 2 · `run_timeout` is declared and read by nothing

Shipped as two hours, validated at parse time, consumed nowhere. Today this is load-bearing in the wrong direction — a standing console is only possible *because* nothing reads it — so it is best understood as a tripwire: if it ever gains a reader, the triage console dies at two hours. The project's own precedent is to delete such fields rather than leave them, on the grounds that a parsed-and-unread field advertises a control that does not exist.

### 3 · Egress is fleet-wide, not per-role

Documented in source and configuration. The blast radius of a compromised or confused worker with egress is the whole allowlist, not its own slice of it.

### 4 · No retry cap

Re-dispatch is bounded by an epoch fence rather than a counter, and that fence is unreachable on the attended path — which is the path every console seat actually runs on.

### Two smaller ones

- The verb-gate ledger is writable by the worker it audits — evidence against a confused agent, not a hostile one. The stronger control is credential scoping.
- A training-detection guard fails *open*: if its process listing fails it produces no matches and therefore no refusal.

Listing these is not self-deprecation. A guardrail inventory containing only working guardrails cannot be audited, because nothing distinguishes a control that was verified from one that was assumed.

## 12 · What transfers

---

Five principles generalise beyond this codebase, and none of them depends on the choice of model or vendor.

### Bound the boundary in bytes, and choose the overflow policy per direction

Refuse outbound, because a half-delivered task is invisible to the agent receiving it. Truncate inbound, because a partial result is still useful — but allocate fairly and report what was cut, or enumeration order silently becomes an editorial decision.

### Treat the agent as an untrusted reporter

Accept its prose; recompute every structural fact host-side. Coverage, membership, evidence and identity should all be derived from state the host owns. This single decision is what makes an unattended verdict worth acting on.

### Declare capability; never infer it

The `hosted` flag is mandatory because every available heuristic misclassified a local endpoint. The same argument applies to trust boundaries, egress grants and write access: a declaration can be reviewed, and an inference cannot.

### Instrument the absence

Prevention leaves no record — a blocked escape attempt and a workload that never tried are indistinguishable in the logs. Build the detector that converts "nothing happened" into evidence, make its three states distinct (armed-and-quiet is not the same as unarmed), and make its death fatal rather than silent.

### Delete configuration nothing reads

A field that parses and validates and is never consumed is worse than a missing feature, because it reads as a control during review. Where such a field is load-bearing by its absence, say so explicitly and treat it as a tripwire.

#### The one-line version

A local fleet is affordable because of what it is **forbidden to send**, not because of what it is asked to do — and it is trustworthy unattended because the host recomputes everything that matters rather than believing the report.

Dan Elliott · cmux-fleet · figures read from the source tree at commit `1959b76`